Sparse Autoencoders for Low-N Protein Function Prediction and Design

Darin Tsui
Georgia Institute of Technology
darint@gatech.edu

Kunal Talreja
Georgia Institute of Technology
ktalreja6@gatech.edu

Amirali Aghazadeh
Georgia Institute of Technology
amiralia@gatech.edu

Abstract

Predicting protein function from amino acid sequence remains a central challenge in data-scarce (low-N) regimes, limiting machine learning–guided protein design when only small amounts of assay-labeled sequence-function data are available. Protein language models (pLMs) have advanced the field by providing evolutionary-informed embeddings and sparse autoencoders (SAEs) have enabled decomposition of these embeddings into interpretable latent variables that capture structural and functional features. However, the effectiveness of SAEs for low-N function prediction and protein design has not been systematically studied. Herein, we evaluate SAEs trained on fine-tuned ESM2 embeddings across diverse fitness extrapolation and protein engineering tasks. We show that SAEs, with as few as 24 sequences, consistently outperform or compete with their ESM2 baselines in fitness prediction, indicating that their sparse latent space encodes compact and biologically meaningful representations that generalize more effectively from limited data. Moreover, steering predictive latents exploits biological motifs in pLM representations, yielding top-fitness variants in 83% of cases compared to designing with ESM2 alone.

1 Introduction

Machine learning (ML)–guided protein engineering seeks to predict and optimize protein function by leveraging evolutionary information and assay-labeled sequence data to model the underlying sequence–function landscape [1–3]. In practice, however, ML models are often constrained by the scarcity of experimental data. Functional assays are costly and time-consuming, so only a small number of variants (low-N) can typically be characterized, creating a fundamental bottleneck for ML-guided design [4–6].

Protein language models (pLMs), trained on large evolutionary sequence datasets, provide embeddings that achieve state-of-the-art performance in zero-shot function prediction [7–9]. These embeddings are widely believed to capture amino acid interactions underlying protein function [10–12], yet they remain difficult to interrogate. More recently, sparse autoencoders (SAEs) have emerged as a powerful interpretability framework, factorizing pLM embeddings into sparse, biologically meaningful latent variables. In high-N regimes (e.g., $N > 800$ labeled sequences), these latents have been shown to align with structural and functional motifs [13–16] and can be steered to design sequences with targeted functional properties [17–19]. Despite these advances, the function prediction and steering performance of SAEs in realistic data-scarce (low-N) settings has not been systematically evaluated. *We hypothesize that the sparse latent space of SAEs, originally introduced as a strategy to*

Preprint. Under review.

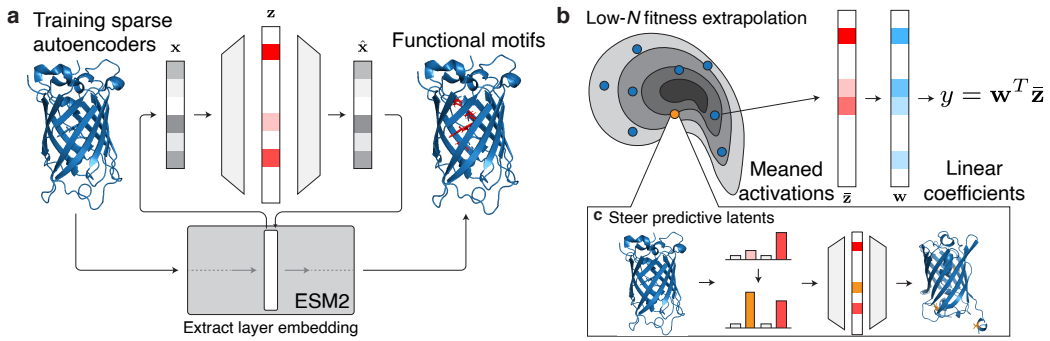

Figure 1: **Overview of downstream low-N tasks for SAEs.** **a**, We train SAEs on the layer embeddings of ESM2. By projecting the model embedding $\mathbf{x}$ to the latent representation $\mathbf{z}$, and reconstructing the model embedding as $\hat{\mathbf{x}}$, the activations in $\mathbf{z}$ correspond to specific biological motifs. **b**, In low-N fitness extrapolation, a linear probe is trained on top of the SAE's latent space to predict protein fitness from N many training sequences. **c**, Using the learned linear probe weights, we steer predictive latents to design highly-functional variants.

enhance interpretability, also encodes compressed and regularized representations that enable accurate fitness prediction and effective protein design from limited data. To test this, we reposition SAEs from proof-of-concept interpretability tools to actionable predictors and design engines, evaluating their performance on downstream protein engineering tasks under low-N conditions. Specifically, we assess their utility across diverse fitness extrapolation challenges that reflect real-world design constraints, and we further examine their ability to design high-functioning variants through latent steering. Our main contributions are as follows:

- We train SAEs on fine-tuned ESM2 embeddings across five proteins with diverse functions.
- We show that SAEs, with as few as 24 sequences, outperform their ESM2 baselines in 58% of fitness extrapolation tasks, while maintaining comparable performance in the remainder.
- We demonstrate that steering SAEs along their most predictive latents produces a diverse pool of highly functional variants, including the top fitness variants in 83% of cases, compared to designing with ESM2 alone.
- We analyze the best-performing steered variants in green fluorescent protein (GFP) and the IgG-binding domain of protein G (GB1), uncovering biologically meaningful motifs that SAEs exploit for steering. All codes and data are available on our GitHub repository `https://github.com/amirgroup-codes/LowNSAE`.

2 Sparse Autoencoders (SAE)

SAEs are autoencoders designed to learn meaningful representations of model embeddings in their latent space (Fig. 1a). We use SAEs with TopK activation [20] to enforce sparsity in the latent space. Given the model embedding $\mathbf{x} \in \mathbb{R}^{d_{\text{model}} \times L}$, where L is the sequence length and d_{model} is the embedding dimension, the encoder maps $\mathbf{x}$ to the SAE latent representation $\mathbf{z} \in \mathbb{R}^{d_{\text{SAE}} \times L}$ via:

$$\mathbf{z} = \text{TopK}(\mathbf{W}_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{pre}})), \tag{1}$$

where $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{d_{\text{SAE}} \times d_{\text{model}}}$ are the encoder weights and $\mathbf{b}_{\text{pre}} \in \mathbb{R}^{d_{\text{model}} \times L}$ is a bias term. The TopK function is applied column-wise to the resulting matrix, keeping only the k largest activations for each of the L sequence positions and setting all other values to zero. The decoder then reconstructs the input $\mathbf{x}$ from $\mathbf{z}$ as:

$$\hat{\mathbf{x}} = \mathbf{W}_{\text{dec}}\mathbf{z} + \mathbf{b}_{\text{pre}}, \tag{2}$$

where $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{SAE}}}$ are the decoder weights. As illustrated in Fig. 1a, where $L = 1$ for simplicity, the activations in $\mathbf{z}$ have been shown to correspond to biological motifs [13, 14].

During training, SAEs minimize both mean squared error and an auxiliary loss. The mean squared error between the original embedding $\mathbf{x}$ and its reconstruction $\hat{\mathbf{x}}$ is defined as $\mathcal{L}_{\text{MSE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$.

To reduce the number of dead latents, defined as latents that never activate [20], an auxiliary loss is included. Given the original reconstruction loss $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$, the auxiliary loss is defined as $\mathcal{L}_{\text{aux}} = \|\mathbf{e} - \hat{\mathbf{e}}\|_2^2$, where $\hat{\mathbf{e}}$ is found by multiplying the decoder matrix by the top-k_{aux} latents in $\mathbf{z}$, where k_{aux} is a hyperparameter. The total SAE training objective, $\mathcal{L}_{\text{SAE}}$, is a weighted sum of these two losses:

$$\mathcal{L}_{\text{SAE}} = \mathcal{L}_{\text{MSE}} + \alpha \mathcal{L}_{\text{aux}},$$

where α is also a hyperparameter. This joint objective enables SAEs to not only reconstruct the original model embeddings faithfully, but also maximize the number of biologically interpretable latents.

3 SAEs for Low-N Fitness Extrapolation

In this section, we first detail the datasets used and how we trained our SAEs. Then, we rigorously evaluate the ability of SAEs to generalize to unseen variants under various low-N regimes (Fig. 1b). To capture the challenges faced in real-world design settings, we define five distinct fitness extrapolation tasks that stress different aspects of the sequence–function landscape: random, position, mutation, regime, and score extrapolation.

3.1 Datasets and SAE Training Details

Datasets. We evaluated our SAEs on six deep mutational scanning (DMS) assays from ProteinGym [21], spanning five distinct proteins (Table 1). These proteins were selected to ensure robust evaluation across a variety of functions. Additionally, these DMS assays also contain multipoint mutations, which are crucial for our fitness extrapolation tasks (see Section 3.2).

Table 1: Summary of DMS assays used.

DMS	Description	Function Tested	Variants	MSA Sequences
GFP_AEQVI_Sarkisyan [22]	Green fluorescent protein	Fluorescence	51,714	396
SPG1_STRSG_Olson [23]	IgG-binding domain of protein G	Binding	536,962	44
SPG1_STRSG_Wu [24]	IgG-binding domain of protein G	Binding	149,360	3,109
DLG4_HUMAN_Faure [25]	Third PDZ domain of PSD95	Yeast growth	6,976	25,338
GRB2_HUMAN_Faure [25]	C-terminal SH3 domain of GRB2	Yeast growth	63,366	33,228
F7YBW8_MESOW_Ding [26]	Antitoxin ParD3	Growth enrichment	7,922	38,613

Training. For each DMS assay, we trained a unique SAE on a fine-tuned ESM2-650M model [7]. Each model was fine-tuned on multiple sequence alignment (MSA) sequences from Table 1 using LoRA adapters, where the MSA sequences were obtained from ProteinGym. Embeddings $\mathbf{x}$ to train the SAE were then obtained by passing the MSA sequences through the fine-tuned model. Following [14], we chose to extract embeddings from layer 24, and set $d_{\text{SAE}} = 4096$, $k = 128$, $\alpha = 1/32$, and $k_{\text{aux}} = 256$. For more details on training, see Appendices A.1 and A.2.

3.2 Experimental Setup

Low-N Regimes. To evaluate the performance of our SAEs and ESM2 in the low-N regime, we first created four distinct N sizes to train a supervised model on top of the SAE latent space and ESM2 embeddings, respectively, to predict fitness: $N \in [8, 24, 96, 384]$. These sizes correspond to standard plate-well sizes used in protein engineering experiments [4].

Fitness Extrapolation Tasks. For each of our DMS assays, we designed five different fitness extrapolation tasks based on ref. [27] to test the ability of our SAE and ESM2 to generalize to unseen variants (see Fig. 2):

1. **Random extrapolation**: We randomly sampled N sequences from the DMS for training and validation, with 10% of the DMS held out as a test set (Fig. 2b).
2. **Mutation extrapolation**: We randomly designated 80% of all possible mutations as training mutations (Fig. 2c). We sampled N sequences for training that only had training mutations. The other 20% of mutations were held out as a test set.